

Chef Rush

A fast-paced mobile cooking game for Android

Prepared by	Aswin	Document	v1.0 — Proposal
Date	9 August 2026	Status	Awaiting approval

Executive summary

Chef Rush is a mobile game in which the player is handed a food order and must assemble it under time pressure. Each round the game names a dish and shows its recipe; the player taps the matching ingredients off a pantry shelf into a mixing bowl, mixes them, and sends the bowl into a microwave oven. The oven reveals the result and the round is scored from zero to three stars on accuracy and speed. Rounds get harder by seeding the shelf with more decoy ingredients, so the difficulty curve comes from content rather than from new code.

The game is built with standard web technology — HTML, CSS and JavaScript, with **three.js** rendering the 3D kitchen — and shipped to Android as a native app using **Capacitor**. The same codebase runs in any browser, so the project is demonstrable from week one and does not depend on a device build to show progress.

At a glance

Platform	Android (Capacitor); playable in any modern browser
Technology	HTML · CSS · JavaScript · three.js · Capacitor
Genre	Casual time-management / reflex puzzle
Session length	2–3 minutes per run, 8 rounds
Effort estimate	6 phases, approx. 6 working weeks, 1 developer
Launch content	10 dishes, ~25 ingredients, expandable by data entry alone

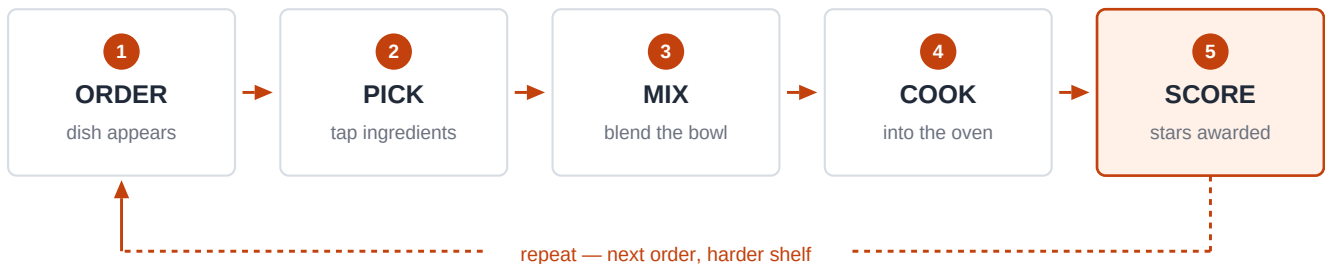
The ask

Approval to proceed with development as scoped in this document, and provisioning of an Android build toolchain (Node.js LTS, JDK 21, Android Studio with SDK 34) on the development machine. The toolchain is required only for the final packaging milestone — development of the game itself can begin immediately upon approval.

1. Concept

The appeal of the cooking-game genre is a simple promise: you are given an order, and you have a few seconds to get it right. Chef Rush reduces that promise to its smallest satisfying form. There is no restaurant to manage, no currency to balance and no menu of upgrades — there is a dish, a shelf of ingredients, a bowl, an oven and a clock. Every second of play is spent on the one decision that matters: *is this the right ingredient?*

The core loop



One round takes roughly fifteen seconds. A full run is eight rounds.

Who it is for

Casual mobile players who play in short bursts — commutes, queues, breaks. The controls are single taps, the rules are explained by the first round rather than by a tutorial, and a run is short enough to finish before an interruption. Because the recipe stays on screen throughout, the game requires no cooking knowledge and no reading speed beyond recognising an ingredient name, which keeps it accessible to a wide age range.

Why this design will hold attention

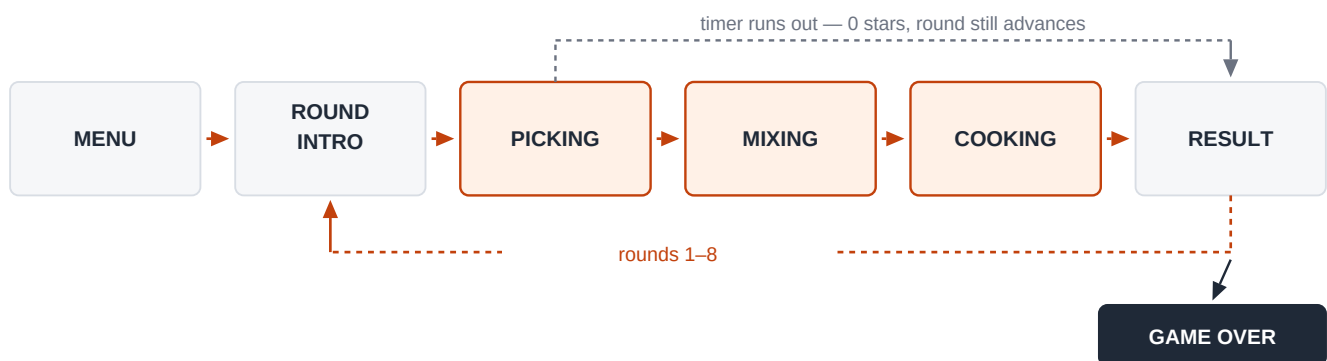
- **The tension is honest.** The recipe is always visible, so a failed round is never the game withholding information — it is the player being too slow or too careless. That reads as fair, and fair failure is what makes players retry.
- **Difficulty scales without new mechanics.** Adding decoy ingredients to the shelf makes a round harder with no new rules to teach and no new code to write.
- **Every action gets a physical response.** Ingredients arc into the bowl and land with a bounce, the bowl spins when mixing, the oven door shuts and glows. The 3D scene exists to make taps feel consequential.
- **A score worth beating.** Star ratings and a persistent high score give a run a target, which is the cheapest and most reliable retention mechanic in the genre.

Scope discipline. Features common to this genre but deliberately excluded from version one: customers and tips, ingredient prep mini-games, in-app purchases, accounts and online leaderboards. Each is a viable follow-up, and each would extend the timeline. The proposal is for a complete, polished small game rather than a partial large one.

2. How a round plays

Phase	What the player does	What the game does
Order	Reads the dish name and its recipe card.	Picks a dish at random, builds the shelf from its real ingredients plus decoys, starts the countdown.
Pick	Taps ingredients on the shelf. Tapping a chosen item again removes it.	Arcs each ingredient into the bowl, ticks it off the recipe card, keeps the clock running.
Mix	Presses MIX once satisfied with the bowl.	Pauses the clock, locks the shelf, spins the bowl and blends the contents into a single batter.
Cook	Sends the bowl into the microwave and waits.	Runs a progress ring, then reveals the plated result and the star rating.

Round state flow



Highlighted states are interactive; the clock runs only during PICKING.

Scoring

The bowl is judged as a set, so the order of picking never matters and duplicates are impossible. The result is broken into ingredients that were *correct*, *wrong*, and *missing* — the same breakdown drives both the score and the feedback shown to the player, so a low score always comes with a visible reason.

Stars	Earned when	Points
★★★	Every ingredient correct, with more than half the time left	Full value plus a time bonus
★★	Every ingredient correct	Full value
★	Most of the recipe right, at most one wrong item	Reduced by a penalty per error
—	Anything less, or the timer expired	None

Difficulty and content

- **Round 1** presents a three-ingredient dish on a shelf with few decoys and a generous clock — effectively a tutorial that never announces itself.
- **By round 8** the dish has up to six ingredients, the shelf is crowded with decoys and the clock is short.
- **Launch content** is ten dishes drawn from roughly twenty-five ingredients. Both live in plain data files: a new dish is one line, a new ingredient is one line. Expanding the game after launch requires no engineering.

3. Technical approach

The game is a single web application wrapped for Android. Rendering responsibilities are split: three.js draws only what benefits from being three-dimensional — the counter, bowl, ingredients and oven — while every menu, button and readout is ordinary HTML sitting above the canvas. This keeps text crisp at any screen density, makes the interface trivial to adjust, and keeps the 3D scene small enough to hold a smooth frame rate on mid-range phones.

Android APK	Capacitor shell — wraps the game in a native WebView
Interface layer	HTML + CSS overlay — recipe card, pantry shelf, HUD, results
Game core	state machine · scoring · dish + ingredient catalogs
Render layer	three.js WebGL scene — bowl, oven, counter, ingredients
Runtime	native ES modules — no bundler, no build step, no Node at runtime

Each layer depends only on the one beneath it.

Three decisions worth flagging

- **No build step.** The project uses the browser's native module system instead of a bundler. There is nothing to compile, so what runs in the browser is exactly what ships in the app, and a developer can open the project and see it running immediately. It also means no build tooling is needed for the first five milestones.
- **Artwork is generated in code, not downloaded.** Ingredients and kitchen objects are assembled from simple 3D shapes with flat shading, giving a consistent stylised look. This removes any dependency on an artist or on third-party asset licensing, keeps the app small, and means a new ingredient costs one line of data rather than a modelling task.
- **Built for touch from the start.** Portrait layout, targets sized for thumbs, safe-area handling for notches and gesture bars, and sound synthesised at runtime rather than streamed from audio files, so the game loads instantly and works offline.

Path to an installable Android app

Capacitor wraps the finished web build in a native Android project. Because the game has no build step, packaging is a copy of the game folder into the Android project followed by a standard Gradle build, producing an installable APK. This step is mechanical and is scheduled as its own milestone; it is also the only part of the project that depends on software not yet installed on the development machine.

4. Delivery plan

#	Milestone	Est.	Demonstrable checkpoint
1	Project scaffold and 3D kitchen	1 wk	The kitchen counter, bowl and oven render on a phone screen.
2	Game data and screen flow	0.5 wk	Dish and ingredient catalogs load; menu, play and results screens navigate.
3	The picking loop	1.5 wk	A full round is playable: shelf, recipe card, tapping, timer.
4	Mix, cook and scoring	1.5 wk	Feature complete — an eight-round run can be played end to end.
5	Polish and progression	1 wk	Sound, feedback, difficulty curve, high scores. Release candidate.
6	Android packaging	0.5 wk	Signed APK installed and tested on a physical device.

Approximately six working weeks for one developer. Milestones 1–5 are independent of the toolchain request below.

Risks and mitigations

Risk	Impact	Mitigation
Build toolchain not installed — the development machine currently has no Node.js, JDK or Android Studio.	Blocks milestone 6 only	The architecture needs none of it for milestones 1–5, so work starts on approval. Provisioning is requested now so it is ready before packaging.
3D performance on low-end phones	Poor frame rate	Deliberately small scene, no shadow rendering, capped resolution, and device testing from milestone 1 rather than at the end.
Content feels thin after a few runs	Weak retention	Dishes and ingredients are data, not code — the catalog can be expanded post-launch without a development cycle.
Scope growth during development	Timeline slip	Excluded features are named explicitly in this document and deferred to a version two rather than absorbed mid-build.

What success looks like

- An APK that installs and runs on a physical Android device, holding a smooth frame rate on mid-range hardware.
- A complete eight-round run playable start to finish with scoring, star ratings and a persistent high score.
- A first-time player completing round one without a tutorial.
- New dishes addable by editing a data file, with no code change.

Approval

Signing below authorises development to begin at milestone 1 and confirms the toolchain provisioning request.

Approved by

Signature

Date